CHAPTER 01.5

Domain-Driven Design & Microservices Architecture

How to split, plan, and design correct microservice boundaries

Bounded Context | DDD | Strangler Fig | Service Boundaries | Anti-Patterns

AGENDA

What We'll Cover

01

Why Planning Matters

The cost of wrong service boundaries — and how to avoid them

02

Domain-Driven Design (DDD)

Bounded contexts, ubiquitous language, and thinking in domains

03

How to Split a Monolith

Identifying service boundaries using real business rules

04

Splitting Patterns

By domain, volatility, scale, and team — with examples

05

The Strangler Fig Pattern

How to migrate gradually without rewriting everything

06

Anti-Patterns to Avoid

Common mistakes that make microservices worse than a monolith

WHY PLANNING MATTERS

The Cost of Wrong Boundaries

Most microservice failures are NOT a technology problem — they're a design problem.

✕ Services too chatty

Service A calls B calls C for every request — worse latency than a monolith

✕ Shared database

Two services, one DB — you just split the code, not the system

✕ Wrong domain boundaries

OrderService handles payments AND notifications — it grows into a mini-monolith

✕ Split too early

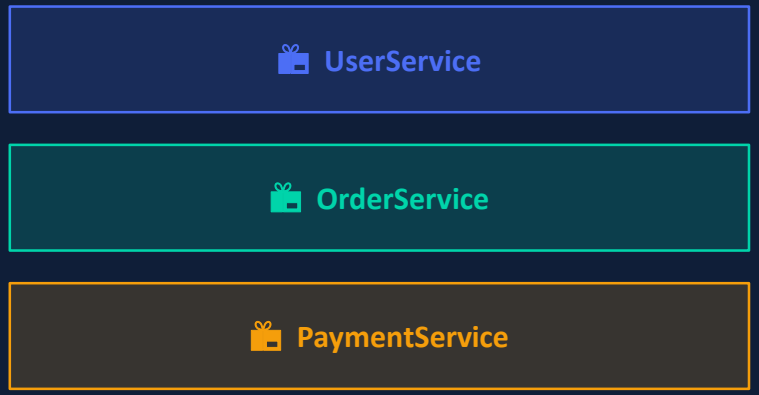
Microservices before you understand the domain = guaranteed redesign in 6 months

✓ The solution: design boundaries around BUSINESS DOMAINS, not technical layers — this is what DDD teaches us.

Core Concepts

Bounded Context

A Bounded Context is the central DDD concept — a clear boundary within which a domain model is defined and applicable.



"Can this part be owned by one small team?"

Ubiquitous Language

Developers and business people use the same words. In OrderService, 'order' means exactly one thing — no confusion, no translation.

Domain vs Sub-domain

Core domain = your competitive advantage (e.g. pricing engine). Supporting domain = needed but not unique (e.g. email). Generic = buy/outsource (e.g. auth).

One Service = One Domain

Each microservice owns exactly one bounded context. Never let two services share domain logic or a database schema.

DOMAIN-DRIVEN DESIGN

Core vs Supporting vs Generic

Not all domains deserve the same engineering effort — DDD tells you where to invest.

BUILD & PROTECT

⚿ Core Domain

Your competitive advantage

What makes customers choose YOU over competitors. This is unique to your business — no one else does it the same way.

MAXIMUM DDD EFFORT

Rich domain models, complex business rules, your best engineers

Examples:

Amazon → Recommendation engine

Wolt → Delivery routing & timing

Spotify → Music discovery algorithm

BUILD SIMPLY

⚿ Supporting Domain

Needed, but not special

You need it to operate, but your competitors do the same thing. Won't make customers choose you — but you still build it because it connects to your core.

MODERATE EFFORT

Simple models, basic CRUD is fine, don't over-engineer

Examples:

Amazon → Order notification system

Wolt → Restaurant menu management

Spotify → Playlist management

BUY / OUTSOURCE

● Generic Domain

Someone solved this already

A commodity problem that hundreds of companies have already solved perfectly. Building your own is a waste of engineering time.

ZERO DDD EFFORT

Use Auth0, SendGrid, Stripe, Twilio — plug it in and move on

Examples:

Amazon → User authentication → Cognito

Wolt → Email notifications → SendGrid

Spotify → Payment processing → Stripe

💡 *Apply FULL DDD only to your Core Domain — that's where business complexity lives and where value is created.*

SERVICE BOUNDARIES

How to Identify Service Boundaries

Ask these 4 questions about every part of your system:

01 What are the main business capabilities?

Not technical functions! Think in terms of what the BUSINESS does. A business places orders, manages inventory, processes payments — not 'CRUD operations on tables'.

02 What changes frequently vs rarely?

If a part of the system changes every sprint (e.g. pricing rules), it should be isolated so changes don't break everything else.

03 What has different scaling needs?

Search gets 10x more traffic than checkout. Product browsing is read-heavy, order placement is write-heavy. Different needs = good candidate for separate service.

04 What can fail independently?

If the Recommendations service goes down, can users still browse and buy? Yes → separate it. If Order always needs User data synchronously → keep them tightly designed.

SPLITTING PATTERNS

4 Ways to Split Your Monolith

By Business Domain

MOST COMMON

The DDD approach. Split along what the business DOES, not how the code is layered.

- → Auth Service
- → Order Service
- → Inventory Service

By Volatility

PRACTICAL

Separate what changes often so frequent deploys don't risk the stable parts.

- → Pricing Engine (changes weekly)
- → Catalog Service (changes rarely)

By Scale

PERFORMANCE

Isolate services that need to scale differently from the rest of the system.

- → Search (read-heavy, scale out)
- → Checkout (write-heavy, different DB)

By Team (Conway's Law)

ORGANIZATIONAL

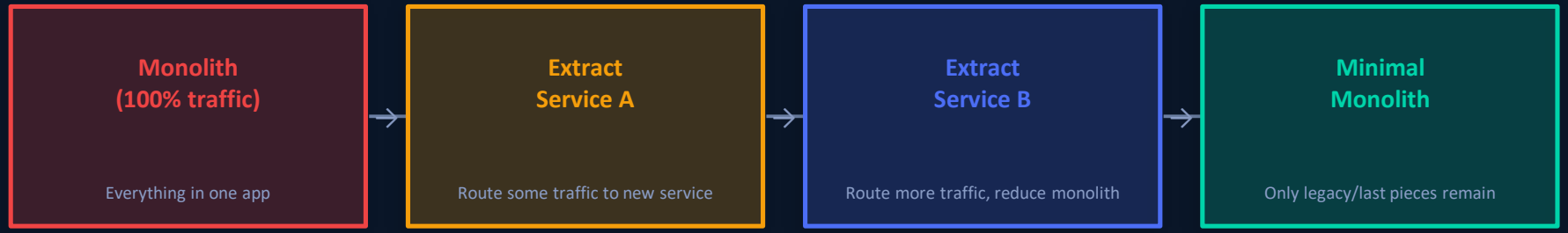
"Organizations ship their org chart." Structure services to match team ownership.

- → Team A owns OrderService
- → Team B owns ProductService

MIGRATION PATTERN

The Strangler Fig Pattern

Don't rewrite everything at once. Extract one service at a time, route traffic gradually.



1. Identify the least risky domain to extract first (e.g. Notifications)
2. Build the new microservice independently with its own database
3. Add a router/gateway that sends some traffic to the new service
4. Test, monitor, gradually increase traffic to the new service
5. Remove the feature from the monolith once the new service is stable
6. Repeat for the next domain — never stop, never big-bang rewrite

WHAT NOT TO DO

Anti-Patterns to Avoid

Split by Technical Layer

❌ Don't create a 'DatabaseService' or 'ValidationService'. Services should own business domains, not technical concerns.

Nanoservices

❌ Services so small they do almost nothing. If UserAddressService calls UserService for every request, they should be one service.

Split Prematurely

❌ Start with a modular monolith. Split when you feel the pain — scaling issues, team conflicts, deployment bottlenecks.

Inseparable Services

❌ If ServiceA and ServiceB always call each other for every operation, they are NOT two services. Merge them back.

Shared Database

❌ Two services, one database = the worst of both worlds. You get distributed system complexity with monolith coupling.

No Bounded Context

❌ Splitting code into multiple repos without thinking about domain ownership. It's still a monolith — just a distributed one.

PRACTICAL EXERCISE

Design the Architecture

Given the Amazon-like monolith below — identify service boundaries. What would you split? Why?

THE MONOLITH

- User Registration & Login
- Product Catalog & Search
- Shopping Cart
- Order Processing
- Payment Handling
- Email Notifications
- Inventory Management
- Admin Dashboard

Your Architecture

1. Which features share a domain?
2. Which scale differently?
3. Which change at different rates?
4. Which can fail independently?
5. Which should share a database?
6. Name each service you create
7. Draw dependencies between them
8. Justify each boundary decision

There is no single right answer — the discussion IS the learning.

Key Takeaways

01 Think in Domains

Design boundaries around what the business DOES, not how the code is technically layered.

04 4 Splitting Patterns

By domain (DDD), by volatility, by scale, by team (Conway's Law).

05 Strangler Fig

Never rewrite everything at once. Extract one service at a time, test, then repeat.

03 4 Boundary Questions

Business capability, rate of change, scaling needs, and failure independence.

Next Chapter: Communication & Core Concepts →